

relazione_CG

Indice

1	Scelte nei punti lasciati all'interpretazione personale	2
1.1	Metrica della classifica	2
1.2	Durata di una partita	2
1.3	Passaggio tra una partita e la successiva	2
1.4	Memorizzazione delle password	2
1.5	Libreria JSON	2
1.6	Codici di errore	2
1.7	Identificatore della partita corrente: il sentinel -1	2
1.8	Persistenza: due file JSON distinti	2
1.9	Ordinamento delle partite (sequenziale ciclico)	2
2	Schema generale dei thread	3
2.1	Lato server	3
2.1.1	Thread 1 — main (1): wiring e accettazione connessioni	3
2.1.2	Thread 2 — scheduler (1): ciclo di gioco	3
2.1.3	Thread 3 — shutdown hook (1): chiusura pulita	3
2.1.4	Thread pool — N ClientHandler (da 0 a N, on-demand)	3
2.2	Lato client	3
2.2.1	Thread 1 — main (1): avvio e loop CLI	4
2.2.2	Thread 2 — udp (1): receiver di notifiche	4
2.3	Sincronizzazione tra i thread del client	4
3	Strutture dati e primitive di sincronizzazione	4
3.1	Lato server	4
3.1.1	Mappe concorrenti	4
3.1.2	Contatori atomici	4
3.1.3	Lock synchronized (lock globale solo dove serve)	5
3.1.4	Sincronizzazione della classifica	5
3.2	Lato client	5
4	Manuale di compilazione ed esecuzione	6
4.1	Struttura del progetto	6
4.2	Compilazione	6
4.2.1	Build e run rapidi con gli script	6
4.3	Esecuzione del server	6
4.4	Esecuzione del client	6

1 Scelte nei punti lasciati all'interpretazione personale

Il bando definisce funzionalità e protocollo, ma lascia aperti alcuni vincoli di progetto. Questo capitolo dichiara, in modo esplicito, come sono state risolte tali scelte nell'implementazione consegnata.

1.1 Metrica della classifica

Il bando richiede una classifica dei giocatori ma non specifica quale valore ordini la graduatoria. È stata scelta la **somma cumulativa dei punteggi di tutte le partite giocate** (`cumulativeScore`), aggiornata a ogni partita conclusa. Un ordine alternativo (es. media, best score, win-rate) avrebbe penalizzato i giocatori meno attivi o premiato il "lotto"; la somma cumulativa è la metrica più semplice e sensata.

1.2 Durata di una partita

Non è fissata dal bando. È **configurabile** tramite `game.duration.sec` in `server.properties` (default pensato sarebbe 600 s); l'assetto consegnato usa 180 s per permettere cicli di verifica in tempi brevi. Lo scheduler, scaduto il tempo, finalizza la partita in modo **idempotente** (una sola volta, anche se richiesto più volte) e procede alla rotazione.

1.3 Passaggio tra una partita e la successiva

Il bando non precisa cosa accade a fine round. È stato scelto che la rotazione **sia immediata** e che i giocatori **online** (traccia in `onlineUsers`) vengano **ri-iscritti automaticamente** alla nuova partita (`GameManager.rotate`). La nuova partita parte senza attendere nuove connessioni; chi si collega dopo l'avvio può comunque partecipare finché la partita corrente non è finita.

1.4 Memorizzazione delle password

Il bando non affronta la sicurezza. Le password sono memorizzate **in chiaro** dentro `users.json`. Scelta consapevole e dichiarata: il progetto è didattico e non tocca temi di autenticazione robusta; in un sistema reale andrebbero usati hash + salt.

1.5 Libreria JSON

È stata adottata **Gson 2.11.0** (jar allegato alla consegna), sia per il server sia per il client.

1.6 Codici di errore

Il protocollo definisce lo *shape* dell'errore ma non il set di codici. I codici sono **testuali** (`ERR_*`), **centralizzati** nell'enum `protocol.Errors` e condivisi da server e client. Su uno stato "vuoto" si usa la convenzione `null = OK`. Questo evita stringhe sparse e garantisce che client e server parlino la stessa lingua di errori.

1.7 Identificatore della partita corrente: il sentinel -1

Il bando lascia aperto come il client richieda la partita "corrente" rispetto a una storica. È stato scelto un **sentinel**: nel campo `Request.roundId`, il valore -1 (default) significa *partita corrente/live*; un valore positivo diverso dall'attuale indica una *partita conclusa* (storico). Il server risolve il sentinel con `currentFor(roundId)`.

1.8 Persistenza: due file JSON distinti

Lo stato persistente è scisso in due file indipendenti:

- `users.json` — credenziali e statistiche dei giocatori;
- `history.json` — storico delle partite concluse (esiti e punteggi).

La separazione rende i due domini (identità vs. storico) salvabili con frequenze e trigger diversi senza interdipendenza.

1.9 Ordinamento delle partite (sequenziale ciclico)

L'insieme di partite in `games.json` viene servito in **ordine sequenziale ciclico**: il loader non carica il file in memoria ma lo scorre in streaming con `JsonReader` a **memoria $O(1)$** , ripetendolo ciclicamente all'esaurimento (`CyclicGameIterator`). Questa scelta evita il costo e i problemi di un indice in RAM e rende il file delle partite semplice da mantenere a mano.

La posizione nel ciclo è inoltre **persistita tra i riavvii**. A ogni partita è assegnato un `roundId` monotono, e la sequenza è deterministica ($0 \dots total-1$ poi wrap): al boot `GameManager.resumeFromRound()` riallinea l'iteratore a $(nextRoundId-1) \% total (= roundId \% total)$, dove `roundId` è il numero di round già giocati, ricostruito in `loadHistory` dal massimo dello storico). Così, dopo uno shutdown e riavvio, si riparte dalla partita **successiva a quella già mostrata**, non dalla prima del file (non fondamentale ma utile in caso di crash causato volontariamente da utenti malevoli).

2 Schema generale dei thread

Le scelte di concorrenza sono separate in due viste indipendenti: il **server**, che accetta più client e coordina la partita, e il **client**, che ne usa solo due per isolare l'ascolto delle notifiche asincrone dal loop dei comandi.

2.1 Lato server

Il server attiva quattro gruppi di thread, uno dei quali a cardinalità variabile (il pool).

Grafico in `server_system_model` (non indispensabile alla comprensione).

2.1.1 Thread 1 — main (1): wiring e accettazione connessioni

Il thread principale (`server.core.ServerMain.main`) svolge i preparativi di avvio: carica la configurazione, costruisce il loader streaming delle partite, il Context con le risorse condivise (store utenti, gestore partite, registro/notifier UDP), avvia lo scheduler e registra lo shutdown hook. Quindi crea il **thread pool** e, senza terminare, entra nel loop bloccante di `ConnectionAcceptor.accept()`. Su ogni nuova connessione TCP il lavoro viene **inviato al pool** (`pool.submit(new ClientHandler(...))`); il thread main non si occupa mai di singoli client.

2.1.2 Thread 2 — scheduler (1): ciclo di gioco

Un singolo thread (`network.GameScheduler`) scandisce la durata della partita. A ogni scadenza esegue: `finalizeGame` (idempotente, una sola finalizzazione), persistenza event-driven di utenti e storico, snapshot dei partecipanti della partita conclusa, quindi la rotazione alla partita successiva con auto-iscrizione dei giocatori online e infine la notifica UDP `notifyEnd` (solo segnale) allo snapshot. L'ordine è intenzionale: quando il client riceve `GAME_ENDED(roundId)`, quel round non è più current, perciò la successiva richiesta TCP legge subito lo storico completo (soluzione ed esito), senza retry TOCTOU. Gestisce anche `InterruptedException` ricalcolando la deadline anziché uscire.

2.1.3 Thread 3 — shutdown hook (1): chiusura pulita

Registrato tramite `Runtime.addShutdownHook`, si attiva su `SIGTERM/SIGINT`. Il suo unico compito è la persistenza finale: salva utenti (`users.persistUsers()`) e storico partite (`persistHistory()`) prima che il processo termini (chiusura pulita del server).

2.1.4 Thread pool — N ClientHandler (da 0 a N, on-demand)

Un `ThreadPoolExecutor` configurato per **non tenere thread inattivi**: core size 0, massimo `pool.size` (default 16, configurabile), keep-alive 10 s, coda **SynchronousQueue**, e una politica di rifiuto **AbortPolicy**. Ogni connessione TCP spawna un worker su richiesta (`pool.submit(new ClientHandler(...))`). In saturazione (tutti i `pool.size` worker occupati) la `SynchronousQueue` (queue di capacità 0, accetta inserimento se è richiesta l'estrazione) non accetta altri task e **AbortPolicy rifiuta**, lanciando `RejectedExecutionException`; l'acceptor intercetta l'eccezione e **chiude il socket** della connessione: i task non si accumulano in coda e il client in eccesso viene disconnesso (può ritentare).

2.2 Lato client

Il client usa esattamente **due thread**.

Grafico in `client_system_model` (non indispensabile alla comprensione).

2.2.1 Thread 1 — main (1): avvio e loop CLI

Il thread principale (`client.ClientMain`) carica la configurazione, apre la connessione TCP NIO, crea il receiver UDP su porta effimera, avvia il thread delle notifiche e infine esegue in prima persona il loop CLI (`Cli.run()`): legge input da `stdin`, costruisce la `Request`, la invia e renderizza la `Response`. Niente pool lato client: un solo thread di interazione.

2.2.2 Thread 2 — udp (1): receiver di notifiche

Un unico thread (`client.UdpClient`, nome "udp") resta in attesa su un `DatagramSocket` per le notifiche asincrone `GAME_ENDED`. All'arrivo richiede una sola volta l'esito storico tramite la connessione TCP e lo stampa con il renderer condiviso: lo scheduler ha già ruotato prima dell'invio, quindi il `roundId` della notifica non coincide con la partita corrente.

2.3 Sincronizzazione tra i thread del client

I due thread client condividono il **medesimo** oggetto `ClientConn`. Il metodo `sendAndRetrieve()` è **synchronized**: un unico lock copre scrittura della richiesta e lettura della risposta, così il thread main e il thread udp non intrecciano mai le righe JSON sul canale. Lo stato di gioco vive sul server; sul client non esiste stato condiviso da proteggere oltre alla connessione.

3 Strutture dati e primitive di sincronizzazione

La maggior parte della sincronizzazione avviene su lato server (usa un grado di parallelismo più alto).

3.1 Lato server

3.1.1 Mappe concorrenti

La base di gran parte dello stato condiviso lato server è `ConcurrentHashMap`.

- `UserStore`: `byId` (`ConcurrentHashMap<Integer, User>`) è la mappa **primaria**, con chiave l'**id immutabile** dell'utente; `nameToId` (`ConcurrentHashMap<String, Integer>`) è l'indice secondario per username. `getById(int)` legge `byId` direttamente (id mai rinominato → risoluzione sempre valida, anche dopo un cambio username in corso di partita).
- `GameManager`: `history` (`ConcurrentHashMap<Integer, GameHistory>`) tiene lo storico delle partite concluse indicizzato per `roundId`; `onlineUsers` è un `ConcurrentHashMap.newKeySet()` (set concorrente) degli utenti loggati, usato per l'auto-join alla rotazione.
- `ActiveGame`: `players` (`ConcurrentHashMap<Integer, PlayerState>`) mappa `userId` → stato di gioco del giocatore.
- `UdpRegistry`: `endpoints` (`ConcurrentHashMap<Integer, Endpoint>`) mappa `userId` → `addr:port` UDP.

Limite della ConcurrentHashMap: garantisce l'atomicità di ciascuna **singola** operazione (`get/put/remove`) ma non offre alcuna atomicità **attraverso più strutture**. Quando un'operazione deve modificare coerentemente *due* strutture condivise, la sola `ConcurrentHashMap` non basta: servono i lock `synchronized` descritti sotto, incluso il lock di metodo su `GameManager`.

3.1.2 Contatori atomici

- `AtomicInteger`: `UserStore.nextId` genera id utente monotoni; `GameManager.nextRoundId` assegna a ogni partita un `roundId` univoco (che non si ripete al ciclo del loader). Al riavvio `loadHistory()` ricostruisce `nextRoundId` dal massimo `roundId` dello storico (e `resumeFromRound()` lo usa per riallineare il ciclo delle partite).
- `AtomicBoolean`: `ActiveGame.finalized` segnala la conclusione della partita. Lettura senza lock nelle validazioni; scrittura unica atomica via `compareAndSet(false, true)` in `finalizeGame` per rendere la finalizzazione **idempotente** (solo la prima procede).
- `volatile`: `GameManager.current` (la partita attiva) è costruita da `makeNext()` prima dell'assegnazione. La `write current = next` è una pubblicazione: nel Java Memory Model essa *happens-before* una successiva `read volatile` che la osserva, quindi il lettore vede l'oggetto correttamente inizializzato, non "a metà". La

lettura lock-free non è però una transazione: concorrendo con `rotate()` può ricevere il vecchio o il nuovo round; l'auto-join avviene dopo lo swap e i dati mutabili usano le proprie primitive (`ConcurrentHashMap`, `AtomicBoolean`, `synchronized(ps)`). In particolare la TOCTOU tra lettura e submit è ammessa, ma sicura: il CAS `finalized` e `finalizeGame` idempotente rifiutano la proposta sul vecchio round. `volatile` non sarebbe sufficiente per proteggere contatori o collezioni mutabili.

3.1.3 Lock `synchronized` (lock globale solo dove serve)

L'approccio evita un unico monitor sull'intero server e adotta granularità variabili, riservando una lock "globale" (per-manager) solo dove serve l'atomicità tra più strutture; altrove si sincronizza finemente, senza mettere un unico monitor sull'intero stato condiviso.

Sul singolo `GameManager` (`synchronized(this)`), è la lock "globale" del manager: serializza le operazioni che devono essere atomiche **attraverso più strutture condivise** (principalmente `onlineUsers` e `current`). È un lock di metodo, quindi copre l'intero `GameManager`; è accettato perché queste operazioni sono rare (login/logout/rotazione/finalizzazione) e non sul percorso caldo delle submit:

- `rotate()`: crea la nuova partita e re-iscrive gli utenti online;
- `registerLogin()` / `registerLogout()` / `logoutUser()`: modifiche ad `onlineUsers` serializzate con `rotate()` per evitare che un login si unisca a una partita appena ruotata o a doppio;
- `finalizeGame()`: idempotente, chiude la partita, scrive storico e aggiorna le statistiche.

Lock granulare su `PlayerState` (`synchronized(ps)`), il cuore del parallelismo: in `ActiveGame.submit()` N client possono presentare proposte **in parallelo**; ogni proposta tocca lo stato del singolo giocatore, quindi il lock è per-giocatore (`ps`), non sulla partita. Lo stesso vale per `wordsOfFoundGroups(ps)`, `outcomeOf(ps)` e la lettura dei contatori in `gameInfo/gameStats`. Un lock di partita (`synchronized(this)` su submit) avrebbe serializzato tutte le proposte: scartato.

Sul singolo `User` (`synchronized(u)`), in `UserStore`, i campi mutabili di un account (password, statistiche) sono protetti dal monitor dell'oggetto `User`. Due utenti diversi non si bloccano mai a vicenda. Le operazioni di mappa (`register`, `login`, `getByName`, `updateCredentials`) si appoggiano a `synchronized(this)` sullo store per la fase di lookup/inserzione; l'aggiornamento dei campi avviene sotto `synchronized(u)`.

Lock dedicato per il disco I/O (`ioLock`), sia `UserStore.persistUsers()` sia `GameManager.persistHistory()` scrivono su un **monitor dedicato** (`ioLock`), separato da `this`. Così l'I/O (lento) non blocca `register/login` o il gameplay; le fonti di persistenza concorrenti si serializzano comunque tra loro su `ioLock`. La scrittura è atomica su file: si scrive su `file.tmp` e poi `Files.move(... ATOMIC_MOVE)`.

3.1.4 Sincronizzazione della classifica

`leaderboard()` non usa un lock globale: cattura uno **snapshot** dei punteggi iterando gli utenti e leggendo `cumulativeScore` sotto `synchronized(u)` per singolo utente. È una "fotografia sfocata"; tra la lettura dei dati di Alice e quelli di Bob `finalizeGame()` potrebbe aggiornare alcune statistiche, ma è accettabile perché la classifica è **read-only** e così si evita un bottleneck di serializzazione con la finalizzazione.

3.2 Lato client

Sul client lo stato condiviso si riduce alla connessione. `ClientConn.sendAndRetrieve()` è **synchronized**: un unico lock copre scrittura della richiesta e lettura della risposta, così il thread main (CLI) e il thread udp (receiver) non intrecciano le righe JSON sul canale TCP.

Il resto dello stato UI (`Cli`) è manipolato solo dal thread main; `UdpClient` non condivide dati con la CLI oltre alla connessione (usa `Cli.renderGameInfo`, statico e stateless). Non esiste quindi nessuna altra struttura sincronizzata lato client.

4 Manuale di compilazione ed esecuzione

4.1 Struttura del progetto

```

Connections_Game/
├── src/                # sorgenti Java
│   ├── client/        # ClientMain, ClientConfig, ClientConn, UdpClient, Cli
│   ├── protocol/      # Request, Response, Errors, GameEnded, payload/*
│   └── server/        # core/, loader/, model/, network/, persistence/
├── lib/gson-2.11.0.jar # dipendenza esterna allegata
├── data/              # games.json (partite), users.json, history.json (a runtime)
├── dist/              # JAR eseguibili (da build.sh / make jar)
├── scripts/           # build.sh, run-server.sh, run-client.sh (build/avvio veloci)
├── server.properties # configurazione server
├── client.properties # configurazione client
└── Makefile           # compile / jar / run / run-client / clean

```

4.2 Compilazione

È richiesto un JDK 8 o superiore (il codice è compilato con `--release 8` come da specifica del corso); il progetto si compila da riga di comando, senza IDE:

```

make          # compila tutto in out/
make jar      # genera anche i JAR eseguibili in dist/

```

I target del Makefile sono: `compile`, `jar`, `run`, `run-client`, `clean`. La dipendenza Gson è già al path `lib/gson-2.11.0.jar` e viene agganciata via `-cp` (e nel Class-Path del manifest dei JAR).

4.2.1 Build e run rapidi con gli script

Nella cartella `scripts/` sono disponibili tre script di uso immediato:

- `scripts/build.sh` - esegue `make clean + make`, compila tutto in `out/` e genera i JAR in `dist/`, con output colorato e conteggio delle classi compilate.
- `scripts/run-server.sh` - avvia il server dal JAR `dist/connections-server.jar` (richiede che l'abbia già generato `build.sh`).
- `scripts/run-client.sh` - avvia il client dal JAR `dist/connections-client.jar` (idem).

Uso tipico: `./scripts/build.sh` la prima volta, poi `./scripts/run-server.sh` per far partire il server e `./scripts/run-client.sh` in un altro terminale per il client.

4.3 Esecuzione del server

Configurazione in `server.properties` (letta all'avvio, nessun parametro interattivo). Avvio (dal JAR, col Main-Class `server.core.ServerMain`):

```

make run          # da Makefile
java -jar dist/connections-server.jar # dal JAR eseguibile

```

4.4 Esecuzione del client

Configurazione in `client.properties`: solo host e port (TCP). La porta UDP **non** è configurabile: è una porta **effimera** scelta a runtime dal receiver e comunicata al server dentro il comando `login`. Avvio:

```

make run-client   # da Makefile
java -jar dist/connections-client.jar # dal JAR eseguibile

```

In generale, sia per client che server, usare gli scripts forniti è più semplice e preferibile.